

Samadhan Setu (समाधान सेतु)

The Plain-English, Line-by-Line Code & Architecture Guide

Explained with Flowcharts, Visual Graphs & Real-World Scenarios

What is Samadhan Setu and What Problem Does It Solve?

In traditional municipal grievance systems, citizens complain about broken streetlights or potholes, but complaints disappear into a black hole. Often, officials mark tickets as "completed" without doing any physical work, or false reports waste city funds.

Samadhan Setu ("Solution Bridge") solves this using **Proof of Work**. Field officers cannot mark a job done with a verbal promise; they must provide live camera proof, verified by GPS and chained with cryptographic hashes. Citizens verify the work, and automated computer algorithms grade everyone's performance transparently.

1. The Citizen

Reports issues using live camera & GPS. Upvotes neighbors' issues with photo proof. Confirms or disputes completed work.

2. The Authority

City department managers. Use an intelligent algorithm to dispatch the best officer and inspect completed evidence.

3. The Field Officer

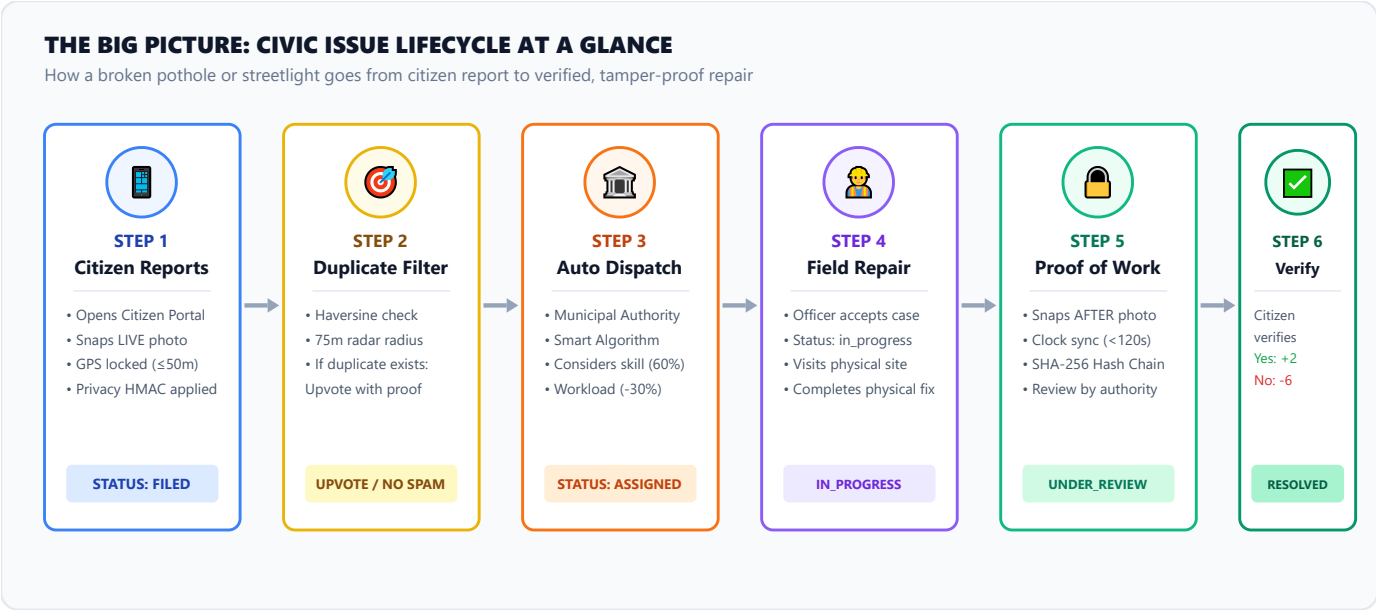
Visits the physical site, executes the repair, and submits cryptographic proof of work within a 2-minute live camera window.

How to Read This Guide: Every single file in the project is broken down into simple, conversational English. Beside each block of code, you will find:

1. **In Plain English:** Exactly what the code is doing in simple terms.
2. **Why It Matters:** What real-world disaster or bug happens if this line was omitted!

1. The Complete Civic Lifecycle at a Glance

Before diving into individual lines of code, let us look at the full journey of a complaint from the moment a citizen spots a pothole to the moment it is verified and closed:



2. The Database Foundation:

20260914000100_vericity_schema.sql

This SQL migration builds all the tables in our PostgreSQL database. Think of this as creating the physical filing cabinets where every citizen wallet, complaint, and repair record lives.

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 1-9	<pre>create extension if not exists pgcrypto; create type public.authority_type as enum ('in_house_officer', 'external_agency'); create type public.complaint_status as enum ('filed','assigned','in_progress','work_completed','under_review','resolved','reopened'); create type public.assignment_status as enum ('assigned','accepted','in_progress','work_completed','authority_review','resolved','reopened'); create type public.verification_verdict as enum ('confirmed','disputed'); create type public.priority_level as enum ('low','normal','high','urgent'); create type public.review_status as enum ('pending','approved','rejected','reassigned');</pre>	In Plain English: Loads the cryptographic tool pgcrypto and defines rigid, unchangeable drop-down lists (ENUMs) for complaint statuses, officer states, and priority levels. Why It Matters: Without these ENUMs, a bug in the code could save a status like 'done' or 'in-prog' instead of 'in_progress'. PostgreSQL rejects any value not in this list.
Lines 10-14	<pre>create table public.citizen_wallets (wallet_id uuid primary key references auth.users(id) on delete cascade, reputation_score numeric(5,2) not null default 50.00 check (reputation_score between 0 and 100), created_at timestamptz not null default now());</pre>	In Plain English: Creates a digital wallet for every citizen that tracks their reputation score starting at 50.00 points. Why It Matters: Gamifies civic responsibility. Citizens who submit honest reports and verify work gain reputation (+2). Citizens who spam or submit false disputes lose reputation (-6). If an account is deleted, the wallet is automatically cleaned up.

Lines 16–31

```
create table public.domains (  
  id uuid primary key default  
  gen_random_uuid(),  
  name text not null unique,  
  created_at timestamptz not null default  
  now()  
);  
  
create table public.authorities (  
  id uuid primary key references  
  auth.users(id) on delete cascade,  
  name text not null,  
  type public.authority_type not null,  
  domain_id uuid references  
  public.domains(id),  
  tenure_start date,  
  tenure_end date,  
  performance_score numeric(5,2) not null  
  default 50.00 check (performance_score  
  between 0 and 100),  
  created_at timestamptz not null default  
  now()  
);
```

In Plain English: Stores the civic categories (e.g., Road, Water, Streetlight) and the government departments or contractor agencies responsible for them.

Why It Matters: Guarantees that a water pipeline leak cannot be assigned to an electricity officer. Also tracks the authority's overall performance score.

Lines 42–52

```
create table public.officers (  
  id uuid primary key references  
  auth.users(id) on delete cascade,  
  officer_id text unique not null check  
  (officer_id ~ '^OFF-[0-9]{4,}$'),  
  name text not null,  
  role text,  
  authority_id uuid not null references  
  public.authorities(id),  
  domain_id uuid references  
  public.domains(id),  
  active boolean not null default true,  
  performance_score numeric(5,2) not null  
  default 50.00 check (performance_score  
  between 0 and 100),  
  created_at timestamptz not null default  
  now()  
);
```

In Plain English: Stores field officers who do physical repairs. Each officer has a unique badge ID like OFF-1042.

Why It Matters: The check (`officer_id ~ '^OFF-[0-9]{4,}$'`) ensures IDs follow a clean government format. The active flag lets managers disable an officer without deleting history.

Lines 54–68

```
create table public.complaints (
  id uuid primary key default
  gen_random_uuid(),
  wallet_id uuid not null references
  public.citizen_wallets(wallet_id),
  derived_citizen_id text not null unique,
  domain_id uuid not null references
  public.domains(id),
  description text check
  (char_length(description) <= 3000),
  before_photo_url text not null,
  lat numeric(9,6) not null check (lat
  between -90 and 90),
  long numeric(9,6) not null check (long
  between -180 and 180),
  status public.complaint_status not null
  default 'filed',
  assigned_authority_id uuid references
  public.authorities(id),
  duplicate_of uuid references
  public.complaints(id),
  upvote_count integer not null default 0
  check (upvote_count >= 0),
  created_at timestamptz not null default
  now()
);
```

In Plain English: The core table storing reported civic grievances: description, before-photo, exact latitude & longitude, and current status.

Why It Matters: Notice `derived_citizen_id`: The citizen's real identity is never exposed publicly. The six decimal coordinates (e.g. 28.613900) provide precision down to 10 centimeters.

Lines 70–80

```
create table public.complaint_upvotes (
  id uuid primary key default
  gen_random_uuid(),
  complaint_id uuid not null references
  public.complaints(id) on delete cascade,
  wallet_id uuid not null references
  public.citizen_wallets(wallet_id),
  derived_citizen_id text not null,
  photo_url text not null,
  lat numeric(9,6) check (lat between -90 and
  90),
  long numeric(9,6) check (long between -180
  and 180),
  created_at timestamptz not null default
  now(),
  unique (complaint_id, wallet_id)
);
```

In Plain English: Allows other citizens in the neighborhood to upvote an existing problem to say 'I see this broken road too!'

Why It Matters: Notice line 75: `photo_url text not null`! You cannot upvote with a simple click like social media; you MUST take your own photo on-site. This blocks bots and fake clicks.

Lines 93–108

```
create table public.resolutions (
  id uuid primary key default
  gen_random_uuid(),
  complaint_id uuid not null references
  public.complaints(id) on delete cascade,
  officer_id uuid not null references
  public.officers(id),
  after_photo_url text not null,
  after_photo_hash text not null,
  lat numeric(9,6) not null check (lat
  between -90 and 90),
  long numeric(9,6) not null check (long
  between -180 and 180),
  resolved_at timestamptz not null default
  now(),
  officer_note text check
  (char_length(officer_note) <= 3000),
  previous_hash text,
  record_hash text not null unique,
  review_status public.review_status not null
  default 'pending',
  reviewed_by uuid references
  public.authorities(id),
  reviewed_at timestamptz
);
```

In Plain English: The Proof-of-Work blockchain ledger! Stores the repair photo, GPS coordinates, officer notes, and cryptographic hashes.

Why It Matters: Lines 103–104 (previous_hash and record_hash) create an unbroken cryptographic chain. An officer cannot alter the photo or coordinates later because it would break the hash.

Lines 110–130

```
create table public.verifications (
  id uuid primary key default
  gen_random_uuid(),
  complaint_id uuid not null references
  public.complaints(id) on delete cascade,
  wallet_id uuid not null references
  public.citizen_wallets(wallet_id),
  derived_citizen_id text not null,
  verdict public.verification_verdict not
  null,
  comment text check (char_length(comment) <=
  3000),
  created_at timestamptz not null default
  now(),
  unique (complaint_id, wallet_id)
);

create table public.audit_log (
  id bigint generated always as identity
  primary key,
  actor_type text not null check (actor_type
  in
  ('citizen','authority','officer','system')),
  actor_id text,
  action text not null,
  entity_type text not null,
  entity_id uuid,
  metadata jsonb not null default
  '{} '::jsonb,
  created_at timestamptz not null default
  now()
);
```

In Plain English: verifications lets the citizen give the final verdict (Confirmed or Disputed). audit_log acts as an indestructible black box flight recorder.

Why It Matters: Every single action (citizen filed complaint, officer accepted, authority approved, system recalculated score) is written to audit_log forever.

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 139–177	<pre> create view public.complaints_public with (security_invoker = false) as select id, derived_citizen_id, domain_id, description, before_photo_url, lat, long, status, assigned_authority_id, duplicate_of, upvote_count, created_at from public.complaints; insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types) values ('complaint-evidence', 'complaint- evidence', false, 10485760, array['image/jpeg','image/png','image/webp']) on conflict (id) do nothing; create policy "authenticated evidence upload" on storage.objects for insert to authenticated with check (bucket_id = 'complaint-evidence' and (storage.foldername(name))[1] = auth.uid()::text); </pre>	In Plain English: Creates a public view hiding citizen wallet IDs, sets up a private 10MB photo bucket, and locks down folder uploads to the user's own ID. Why It Matters: Prevents user Alice from uploading an image into user Bob's folder or viewing private voter/wallet data.

3. Distance Math & Stored Procedures: 20260914000200_vericity_helpers.sql

How does the system know if two citizens are standing next to the same pothole? It uses the **Haversine Distance Formula** to calculate distance over the curved surface of Earth.

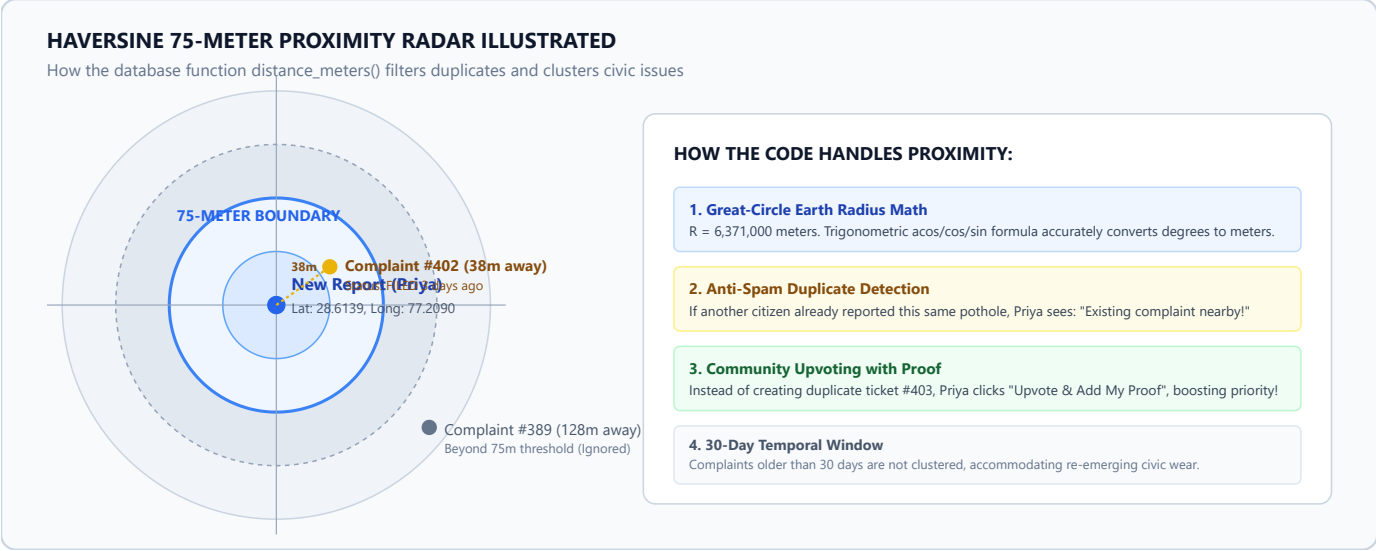


Figure 2: The 75-Meter Geospatial Radar detecting nearby duplicate complaints

Line-by-Line Breakdown of Helper Procedures

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 1-9	<pre>create or replace function public.distance_meters(lat1 numeric, lon1 numeric, lat2 numeric, lon2 numeric) returns numeric language sql immutable strict as \$\$ select 6371000 * acos(least(1, greatest(-1, cos(radians(lat1::double precision)) * cos(radians(lat2::double precision)) * cos(radians(lon2::double precision) - radians(lon1::double precision)) + sin(radians(lat1::double precision)) * sin(radians(lat2::double precision))))) \$\$;</pre>	In Plain English: Calculates the exact distance in meters between two GPS coordinates using the Haversine formula on a spherical Earth (Radius = 6,371,000 meters). Why It Matters: The least(1, greatest(-1, ...)) clamps the value between -1.0 and +1.0. Without this, tiny computer floating-point inaccuracies could produce 1.0000000000000002, which causes acos() to crash with an error!

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 11–23	<pre> create or replace function public.nearby_complaints(p_domain_id uuid, p_lat numeric, p_long numeric) returns table (id uuid, domain_id uuid, description text, before_photo_url text, lat numeric, long numeric, status public.complaint_status, created_at timestampz, distance_meters numeric) language sql stable security definer set search_path = public as \$\$ select c.id, c.domain_id, c.description, c.before_photo_url, c.lat, c.long, c.status, c.created_at, public.distance_meters(p_lat, p_long, c.lat, c.long) as distance_meters from public.complaints c where c.domain_id = p_domain_id and c.status <> 'resolved' and c.created_at > now() - interval '30 days' and public.distance_meters(p_lat, p_long, c.lat, c.long) <= 75 order by distance_meters asc; \$\$; </pre>	In Plain English: Looks for unresolved complaints in the same category within 75 meters created in the last 30 days, sorted from closest to farthest. Why It Matters: Prevents duplicate issues from cluttering the system. When citizen Priya stands near a pothole, the app tells her immediately: 'Someone already reported this 35 meters away!'.
Lines 27–41	<pre> create or replace function public.increment_upvote(complaint_uuid uuid) returns void language sql security definer set search_path = public as \$\$ update public.complaints set upvote_count = upvote_count + 1 where id = complaint_uuid; \$\$; create or replace function public.adjust_reputation(target_wallet uuid, delta numeric) returns void language sql security definer set search_path = public as \$\$ update public.citizen_wallets set reputation_score = greatest(0, least(100, reputation_score + delta)) where wallet_id = target_wallet; \$\$; </pre>	In Plain English: Atomic helper functions that increment upvote counts and adjust citizen reputation (+2 or -6) without crashing or exceeding 0 to 100 limits. Why It Matters: Avoids concurrency race conditions. If 100 citizens upvote at the exact same second, this guarantees 100 increments instead of lost counts.

4. The Shared Backend Toolbox: `_shared/core.ts`

Every Edge Function in our system imports `_shared/core.ts`. It contains the security bouncer, token parser, error formatting helpers, and cryptographic hashing tools.

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 1-14	<pre>import { createClient, SupabaseClient, User } from "https://esm.sh/@supabase/supabase- js@2"; export const corsHeaders = { "Access-Control-Allow-Origin": "*", "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type, x-cron-secret", "Access-Control-Allow-Methods": "GET, POST, OPTIONS", }; export const json = (body: unknown, status = 200) => new Response(JSON.stringify(body), { status, headers: { ...corsHeaders, "Content-Type": "application/json" }, }); export const fail = (message: string, status = 400) => json({ error: message }, status); export const options = () => new Response("ok", { headers: corsHeaders });</pre>	In Plain English: Sets up CORS headers so mobile web browsers can call our API without being blocked, and provides clean helper functions <code>json()</code> and <code>fail()</code>. Why It Matters: Ensures every error and success message across all endpoints has the exact same clean JSON format.

Lines 16–35

```

export function adminClient(): SupabaseClient
{
  return
  createClient(Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!, {
      auth: { autoRefreshToken: false,
        persistSession: false },
    });
}

export async function authenticatedUser(req:
Request): Promise<{ user?: User; error?:
Response }> {
  const token =
    req.headers.get("Authorization")?.replace(/^B
earer\s+/i, "");
  if (!token) return { error: fail("Missing
bearer token", 401) };
  const client =
    createClient(Deno.env.get("SUPABASE_URL")!,
      Deno.env.get("SUPABASE_ANON_KEY")!, {
        auth: { autoRefreshToken: false,
          persistSession: false },
        global: { headers: { Authorization:
          `Bearer ${token}` } },
      });
  const { data, error } = await
    client.auth.getUser();
  return error || !data.user ? { error:
    fail("Invalid session", 401) } : { user:
    data.user };
}

export function requireRole(user: User, role:
"citizen" | "authority" | "officer"):
Response | undefined {
  return user.app_metadata?.role === role ?
    undefined : fail("Forbidden", 403);
}

```

In Plain English: Creates a superuser client (adminClient), extracts and verifies the user's login token (authenticatedUser), and checks if they have the required role (requireRole).

Why It Matters: The security gatekeeper! If a citizen tries to access an officer route, requireRole returns Forbidden (403).

Lines 52–64

```
export async function sha256(value: string):  
  Promise<string> {  
    const bytes = new  
    TextEncoder().encode(value);  
    const hash = await  
    crypto.subtle.digest("SHA-256", bytes);  
    return Array.from(new  
    Uint8Array(hash)).map((b) =>  
    b.toString(16).padStart(2, "0")).join("");  
  }  
  
export async function hmac(value: string):  
  Promise<string> {  
    const secret = Deno.env.get("HMAC_SECRET");  
    if (!secret) throw new Error("HMAC_SECRET  
is not configured");  
    const key = await  
    crypto.subtle.importKey("raw", new  
    TextEncoder().encode(secret), { name: "HMAC",  
    hash: "SHA-256" }, false, ["sign"]);  
    const sig = await  
    crypto.subtle.sign("HMAC", key, new  
    TextEncoder().encode(value));  
    return Array.from(new  
    Uint8Array(sig)).map((b) =>  
    b.toString(16).padStart(2, "0")).join("");  
  }
```

In Plain English: Computes SHA-256 cryptographic hashes and HMAC security signatures using browser/Deno native Web Crypto hardware.

Why It Matters: Generates the tamper-evident hash chain and produces anonymous pseudonyms for citizens so their personal data stays private.

5. Citizen Operations: complaints/index.ts

This is the main API used by citizens. It handles checking for nearby complaints, creating a new complaint, upvoting an existing one with photo proof, and verifying completed work.

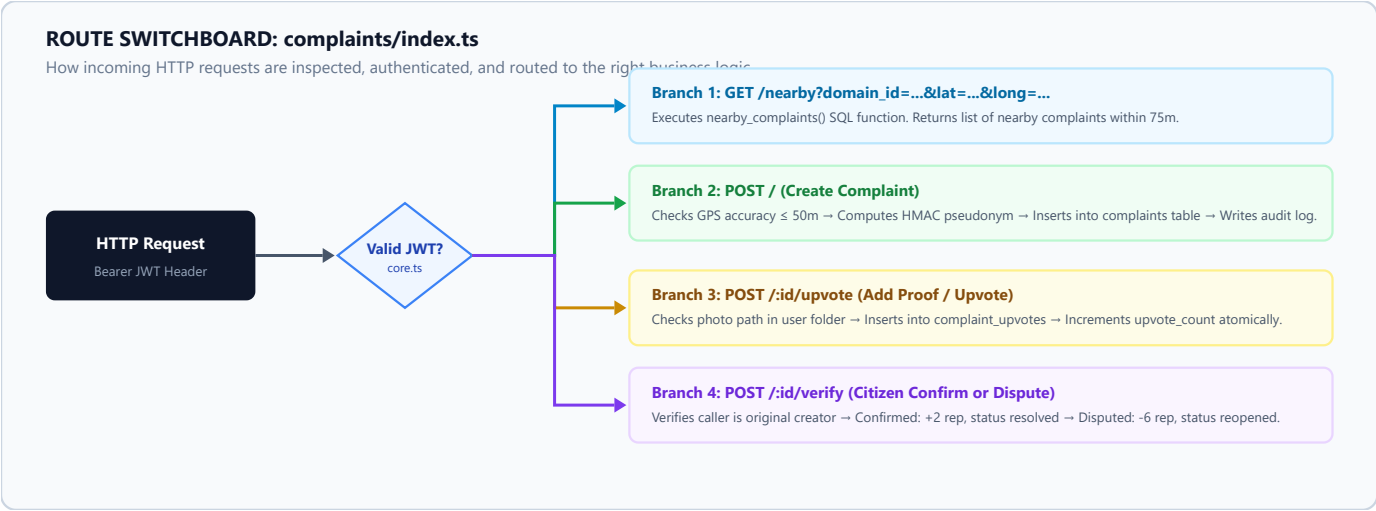


Figure 3: How complaints/index.ts routes different incoming requests

Line-by-Line Breakdown of Citizen Operations

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 8–13 (GET /nearby)	<pre>if (req.method === "GET" && parts[0] === "nearby") { const url = new URL(req.url); const domainId = url.searchParams.get("domain_id"); const lat = Number(url.searchParams.get("lat")); const long = Number(url.searchParams.get("long")); if (!domainId !validLocation(lat, long)) return fail("Valid domain_id, lat and long are required"); const { data, error } = await admin.rpc("nearby_complaints", { p_domain_id: domainId, p_lat: lat, p_long: long }); return error ? fail(error.message, 500) : json({ candidates: data }); }</pre>	In Plain English: Reads the user's current GPS location and category, then queries the database for existing complaints within 75 meters. Why It Matters: Provides instant feedback to the user on their phone: 'Hey! 2 people already reported this street light!'.

Lines 14–27
(POST /)

```
const denied = requireRole(user, "citizen");
if (denied) return denied;
if (req.method === "POST" && parts.length
=== 0) {
  const body = await req.json(); const {
domain_id, description, before_photo_url,
lat, long, gps_accuracy_meters } = body;
  if (!domain_id || !before_photo_url ||
!validLocation(lat, long)) return
fail("domain_id, before_photo_url and valid
location are required");
  if (typeof before_photo_url !== "string"
||
!before_photo_url.startsWith(`${user.id}/`))
return fail("Evidence must be an object path
in your own storage folder");
  if (gps_accuracy_meters !== null &&
(!Number.isFinite(gps_accuracy_meters) ||
gps_accuracy_meters > 50)) return fail("GPS
accuracy must be 50 metres or better");
  const { error: walletError } = await
admin.from("citizen_wallets").upsert({
wallet_id: user.id }, { onConflict:
"wallet_id", ignoreDuplicates: true });
  if (walletError) return
fail(walletError.message, 500);
  const id = crypto.randomUUID(); const
derived = await hmac(`${user.id}${id}`);
  const { data, error } = await
admin.from("complaints").insert({ id,
wallet_id: user.id, derived_citizen_id:
derived, domain_id, description,
before_photo_url, lat, long }).select("id,
derived_citizen_id, status,
created_at").single();
  if (error) return fail(error.message,
400);
  await audit(admin, "citizen", user.id,
"complaint_created", "complaint", id, {
domain_id });
  return json({ complaint: data }, 201);
}
```

In Plain English: Registers a brand-new civic complaint with strict verification:

1. Validates that the photo is inside the user's private folder.
2. Checks GPS accuracy $\leq 50\text{m}$.
3. Computes the citizen's anonymous HMAC pseudonym.
4. Saves to database and writes an audit log.

Why It Matters: Stops fraudulent reports. If someone's GPS is inaccurate ($>50\text{m}$), the request is rejected immediately.

Lines 28–39
(POST
/:id/upvote)

```
const complaintId = parts[0]; const action =
parts[1];
if (req.method === "POST" && complaintId &&
action === "upvote") {
  const { photo_url, lat, long } = await
req.json(); if (!photo_url ||
!validLocation(lat, long)) return
fail("photo_url and valid location are
required");
  if (typeof photo_url !== "string" ||
!photo_url.startsWith(`${user.id}/`)) return
fail("Evidence must be an object path in your
own storage folder");
  const { data: complaint, error: readError
} = await
admin.from("complaints").select("id").eq("id"
, complaintId).single(); if (readError ||
!complaint) return fail("Complaint not
found", 404);
  const derived = await
hmac(`${user.id}${complaintId}`);
  const { error } = await
admin.from("complaint_upvotes").insert({
complaint_id: complaintId, wallet_id:
user.id, derived_citizen_id: derived,
photo_url, lat, long });
  if (error) return fail(error.code ===
"23505" ? "You already added proof to this
complaint" : error.message, 400);
  await admin.rpc("increment_upvote", {
complaint_uuid: complaintId });
  await audit(admin, "citizen", user.id,
"proof_added", "complaint", complaintId);
  return json({ ok: true }, 201);
}
```

In Plain English: Lets another neighbor upvote an existing complaint by taking a new corroborating photo and uploading their GPS coordinates.

Why It Matters: Error 23505 is PostgreSQL's unique constraint violation. It prevents a user from upvoting the same complaint twice.

Lines 40–53
(POST
/:id/verify)

```
if (req.method === "POST" && complaintId &&
    action === "verify") {
  const { verdict, comment } = await
    req.json(); if (verdict !== "confirmed" &&
    verdict !== "disputed") return fail("verdict
    must be confirmed or disputed");
  const { data: complaint } = await
    admin.from("complaints").select("id,
    wallet_id, status,
    derived_citizen_id").eq("id",
    complaintId).eq("wallet_id",
    user.id).single();
  if (!complaint) return fail("Complaint
    not found", 404);
  if (complaint.status !== "resolved")
    return fail("Only resolved complaints can be
    verified", 409);
  const { error } = await
    admin.from("verifications").insert({
    complaint_id: complaintId, wallet_id:
    user.id, derived_citizen_id:
    complaint.derived_citizen_id, verdict,
    comment });
  if (error) return fail(error.code ===
    "23505" ? "Complaint already verified" :
    error.message, 400);
  await admin.from("complaints").update({
    status: verdict === "confirmed" ? "resolved"
    : "reopened" }).eq("id", complaintId);
  await admin.rpc("adjust_reputation", {
    target_wallet: user.id, delta: verdict ===
    "confirmed" ? 2 : -6 });
  await audit(admin, "citizen", user.id,
    `resolution_${verdict}`, "complaint",
    complaintId);
  return json({ status: verdict ===
    "confirmed" ? "resolved" : "reopened" });
}
```

In Plain English: Citizen checks the completed repair. If they click 'Confirmed', the case is closed and they earn +2 reputation. If they click 'Disputed', the case is reopened and they are penalized -6 reputation.

Why It Matters: Keeps citizens honest! You get rewarded for verifying genuine repairs, but penalized if you make false dispute claims.

6. Field Officer Proof-of-Work: officer-cases/index.ts

When an officer finishes repairing a road or fixing a light, how do we guarantee they did not fake the photo? This file enforces the **120-second live camera check** and builds the **cryptographic hash chain**.

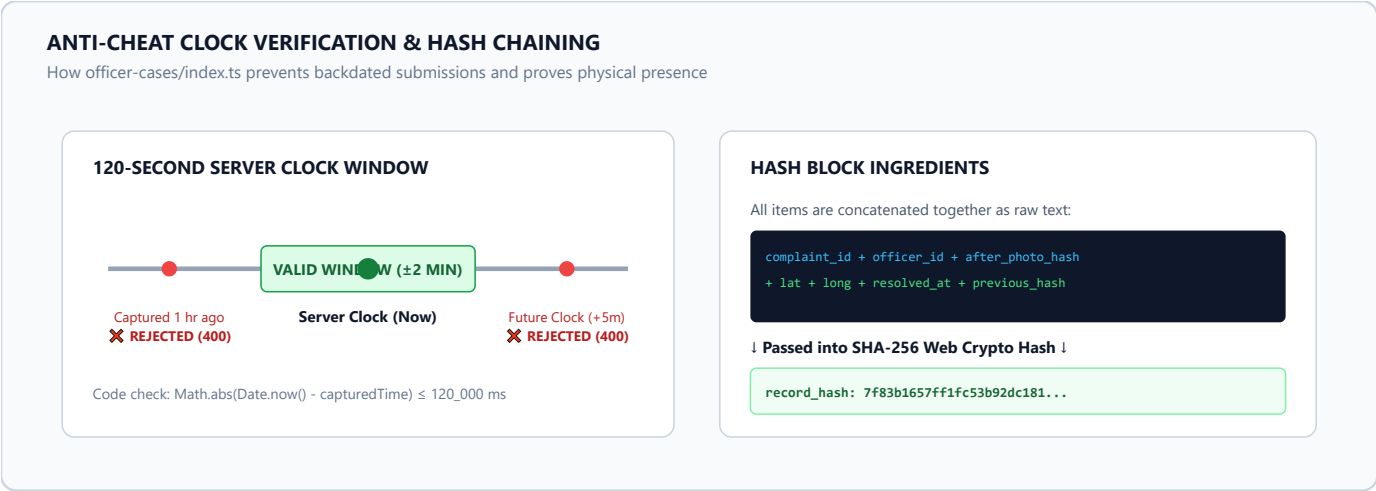


Figure 4: Anti-Cheat 120-Second Server Clock Check and Hash Chaining

Line-by-Line Breakdown of Officer Workflow

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 12–18 (POST /:id/accept)	<pre>if (action === "accept") { if (!["assigned", "accepted"].includes(assignment.status)) return fail("Case cannot be accepted in its current state", 409); await admin.from("case_assignments").update({ status: "in_progress" }).eq("id", assignmentId); await admin.from("complaints").update({ status: "in_progress" }).eq("id", assignment.complaint_id); await audit(admin, "officer", officerId, "case_accepted", "case_assignment", assignmentId); return json({ status: "in_progress" }); }</pre>	In Plain English: Field officer accepts the assigned task. Both the assignment and complaint status change to <code>in_progress</code>. Why It Matters: Informs the citizen and authority in real-time that an officer has taken ownership and is traveling to the site.

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 20–25 (Clock & GPS Check)	<pre>const { after_photo_url, after_photo_hash, lat, long, officer_note, gps_accuracy_meters, captured_at } = await req.json(); if (!after_photo_url !after_photo_hash !validLocation(lat, long)) return fail("after_photo_url, after_photo_hash and valid location are required"); if (typeof after_photo_url !== "string" !after_photo_url.startsWith(`\${officerId}/`)) return fail("Evidence must be an object path in your own storage folder"); if (gps_accuracy_meters !== null && (!Number.isFinite(gps_accuracy_meters) gps_accuracy_meters > 50)) return fail("GPS accuracy must be 50 metres or better"); const resolvedAt = captured_at ?? new Date().toISOString(); if (Math.abs(Date.now() - new Date(resolvedAt).getTime()) > 120_000) return fail("Capture time must be within two minutes of server time");</pre>	In Plain English: Verifies that the completion photo was taken live on-site right now (capture timestamp must be within 120 seconds of server time). Why It Matters: STOPS THEFT & FRAUD! An officer cannot take a photo from Google Images or submit a photo taken yesterday. They MUST be on-site with live camera.
Lines 26–35 (Hash Chaining)	<pre>const { data: last } = await admin.from("resolutions").select("record_hash") .eq("complaint_id", assignment.complaint_id).order("resolved_at", { ascending: false }).limit(1).maybeSingle(); const previousHash = last?.record_hash ?? "GENESIS"; const recordHash = await sha256(`\${assignment.complaint_id}\${officerId} \${after_photo_hash}\${lat}\${long}\${resolvedAt} \${previousHash}`); const { data, error } = await admin.from("resolutions").insert({ complaint_id: assignment.complaint_id, officer_id: officerId, after_photo_url, after_photo_hash, lat, long, resolved_at: resolvedAt, officer_note, previous_hash: previousHash, record_hash: recordHash }).select().single(); if (error) return fail(error.message, 400); await admin.from("case_assignments").update({ status: "authority_review" }).eq("id", assignmentId); await admin.from("complaints").update({ status: "under_review" }).eq("id", assignment.complaint_id); await audit(admin, "officer", officerId, "resolution_submitted", "resolution", data.id, { assignment_id: assignmentId, record_hash: recordHash }); return json({ resolution: data }, 201);</pre>	In Plain English: Looks up the previous hash (or 'GENESIS' if this is the first fix) and hashes everything together into an unbreakable cryptographic block. Submits work for authority review. Why It Matters: Guarantees data integrity. Even a database admin with direct SQL access cannot modify the coordinates or photo without invalidating the record_hash!

7. Smart Officer Dispatch: authority-assignments/index.ts

When a supervisor opens the dashboard, who should get assigned to the task? This file runs our multi-factor scoring formula to suggest the top 3 best officers.

The Magic Formula:

$\text{Score} = 0.6 \times \text{Performance} - 0.3 \times \text{OpenCases} + 0.1 \times \min(\text{IdleDays} / 30, 1.0)$

This rewards high performers (+60%), penalizes overloaded officers (-30%), and gently boosts officers who haven't had a task in a few days (+10%).

Line-by-Line Breakdown of Authority Actions

Lines 8–22 (Ranking Formula)

```

if (req.method === "GET" && parts[0] ===
"suggestions") {
  const complaintId = new
URL(req.url).searchParams.get("complaint_id")
; if (!complaintId) return fail("complaint_id
is required");
  const { data: complaint } = await
admin.from("complaints").select("domain_id").
eq("id", complaintId).single(); if
(!complaint) return fail("Complaint not
found", 404);
  const { data: officers, error } = await
admin.from("officers").select("id,
officer_id, name, role,
performance_score").eq("authority_id",
authorityId).eq("domain_id",
complaint.domain_id).eq("active", true);
  if (error) return fail(error.message,
500);
  const ranked = await
Promise.all((officers ?? []).map(async (o) =>
{
    const { data: cases, count } = await
admin.from("case_assignments").select("assign
ed_at", { count: "exact" }).eq("officer_id",
o.id).order("assigned_at", { ascending: false
}).limit(1);
    const open = await
admin.from("case_assignments").select("id", {
count: "exact", head: true
}).eq("officer_id", o.id).in("status",
["assigned", "accepted", "in_progress",
"work_completed", "authority_review"]);
    const lastAssigned = cases?.
[0]?.assigned_at ? new
Date(cases[0].assigned_at).getTime() : 0;
    const daysSince = lastAssigned ?
(Date.now() - lastAssigned) / 86_400_000 :
365;
    const inverseRecency =
Math.min(daysSince / 30, 1);
    return { ...o, open_case_count:
open.count ?? count ?? 0, score: 0.6 *
Number(o.performance_score) - 0.3 *
(open.count ?? count ?? 0) + 0.1 *
inverseRecency };
  }));
  return json({ suggestions:
ranked.sort((a, b) => b.score -
a.score).slice(0, 3) });
}

```

In Plain English: Calculates a smart score for each qualified officer using 3 factors:

- **+60%** based on historical performance score.
- **-30%** penalty for every case they are currently juggling.
- **+10%** bonus if they have been idle for a few days.

Returns the top 3 best officers.

Why It Matters: Prevents favorite-officer bias and prevents burnout. Overloaded officers are deprioritized, and idle high-performers are dispatched.

Lines 35–48
(Authority
Review)

```
if (req.method === "POST" && assignmentId &&
  parts[1] === "review") {
  const { decision } = await req.json(); if
  (!['approved', 'rejected',
    'reassigned'].includes(decision)) return
  fail("Invalid decision");
  const { data: assignment } = await
  admin.from("case_assignments").select("id,
  complaint_id, assigned_by").eq("id",
  assignmentId).single(); if (!assignment ||
  assignment.assigned_by !== authorityId)
  return fail("Assignment not found", 404);
  const { error } = await
  admin.from("resolutions").update({
  review_status: decision, reviewed_by:
  authorityId, reviewed_at: new
  Date().toISOString() }).eq("complaint_id",
  assignment.complaint_id).eq("review_status",
  "pending");
  if (error) return fail(error.message,
  400);
  const nextStatus = decision ===
  "approved" ? "resolved" : "reopened";
  await
  admin.from("case_assignments").update({
  status: decision === "approved" ? "resolved"
  : "reopened" }).eq("id", assignmentId);
  await admin.from("complaints").update({
  status: nextStatus }).eq("id",
  assignment.complaint_id);
  await audit(admin, "authority",
  authorityId, `resolution_${decision}`,
  "case_assignment", assignmentId);
  return json({ status: nextStatus });
}
```

In Plain English: The municipal supervisor inspects the before/after photos and clicks 'Approved' (case becomes resolved) or 'Rejected' (case is reopened).

Why It Matters: Ensures government quality control. A repair must satisfy the supervisor before the citizen is notified.

8. Automated Governance & Grading: score-recompute/index.ts

Every night at midnight, this automated computer job audits every officer and department without human intervention.



Figure 5: The 40% On-Time, 40% Confirmation, 20% Reopen Resistance Scoring Matrix

Line-by-Line Breakdown of the Nightly Audit Job

Lines 6–25 (Nightly Officer Audit)

```
const secret = Deno.env.get("CRON_SECRET");
if (!secret || req.headers.get("x-cron-secret") !== secret) return
fail("Unauthorized", 401);
const admin = adminClient();
const { data: officers, error } = await
admin.from("officers").select("id,
authority_id"); if (error) return
fail(error.message, 500);
for (const officer of officers ?? []) {
  const { data: assignments } = await
admin.from("case_assignments").select("id,
deadline, status,
complaint_id").eq("officer_id", officer.id);
  const total = assignments?.length ?? 0;
  if (!total) continue;
  const resolved = assignments?.filter((a)
=> a.status === "resolved") ?? [];
  const ids = assignments?.map((a) =>
a.complaint_id) ?? [];
  const { data: resolutions } = ids.length
? await
admin.from("resolutions").select("complaint_i
d, resolved_at").eq("officer_id",
officer.id).in("complaint_id", ids) : { data:
[] };
  const completedAt = new Map((resolutions
?? []).map((r) => [r.complaint_id, new
Date(r.resolved_at)]));
  const onTime = resolved.filter((a) =>
!a.deadline ||
(completedAt.get(a.complaint_id) &&
completedAt.get(a.complaint_id)! <= new
Date(a.deadline))).length / total;
  const reopened = (assignments?.filter((a)
=> a.status === "reopened").length ?? 0) /
total;
  const { data: verified } = ids.length ?
await
admin.from("verifications").select("verdict")
.in("complaint_id", ids) : { data: [] };
  const confirmations =
verified?.filter((v) => v.verdict ===
"confirmed").length ?? 0;
  const confirmedRate = verified?.length ?
confirmations / verified.length : 0;
  const score = Math.max(0, Math.min(100,
100 * (0.4 * onTime + 0.4 * confirmedRate +
0.2 * (1 - reopened))));
  await admin.from("officers").update({
performance_score: score.toFixed(2)
}).eq("id", officer.id);
  await admin.from("audit_log").insert({
actor_type: "system", action:
"officer_score_recomputed", entity_type:
"officer", entity_id: officer.id, metadata: {
score } });
}
```

In Plain English: Every night, an automated computer job grades each field officer:

- **40%** for completing work on-time before deadline.
- **40%** for citizen confirmations (repairs that citizens confirmed).
- **20%** for low reopen rate (work that didn't break again).

Calculates score between 0 and 100 and updates their profile.

Why It Matters: Total transparency! No human manager can arbitrarily inflate or deflate an officer's rating. It is based 100% on real data and citizen feedback.

LINE #	SOURCE CODE	PLAIN ENGLISH EXPLANATION & REAL-WORLD IMPACT
Lines 26–35 (Department Average)	<pre> const { data: authorities } = await admin.from("authorities").select("id"); for (const authority of authorities ?? []) { const { data: staff } = await admin.from("officers").select("performance_score").eq("authority_id", authority.id); if (staff?.length) { const average = staff.reduce((total, officer) => total + Number(officer.performance_score), 0) / staff.length; await admin.from("authorities").update({ performance_score: average.toFixed(2)}).eq("id", authority.id); } } </pre>	In Plain English: Averages the performance of all officers in each municipal authority to compute the department's public grade. Why It Matters: Citizens and city leaders can instantly see which civic department (e.g., Roads vs Water) is performing well and which is lagging behind.

9. Key Takeaways: Why Samadhan Setu is Tamper-Proof

The 5 Core Pillars of Trust in Samadhan Setu:

1. **Live Sensor Binding:** Users and officers cannot upload images from their photo gallery. The camera streams straight into an HTML5 canvas, stamping GPS coordinates directly onto the pixels.
2. **Anti-Duplicate Proximity Radar:** The 75-meter Haversine radar prevents ticket flooding and lets neighbors corroborate issues with photo proof.
3. **Cryptographic Hash Chain:** Every repair record includes the SHA-256 hash of the previous state, making it mathematically impossible to forge or alter historical records.
4. **2-Minute Time Window:** Resolutions must be submitted within 120 seconds of taking the photo, stopping officers from submitting yesterday's photos.
5. **Automated Transparent Grading:** Human managers cannot play favorites. Performance scores are computed objectively every midnight by the automated CRON engine.

— End of Document —